```
import type { Coach, Diagram } from './services/sinucaCoach.js';
import type { WhatsAppClient } from './services/whatsapp.js';
import { MediaError, validateImage, type SupportedMime } from './services/media.j
import { MSG_FREE_HINT, MSG_PAID, type Paywall } from './services/paywall.js';
import { choicePart, SPOKEN_PHOTO_WAIT, type Voice } from './services/voice.js';
import { errorInfo, logger, maskPhone } from './utils/logger.js';

export const MSG_TEXT =
  'Fala! 🎱 Sou o treinador de sinuca em fase de testes. Manda uma foto da mesa (
export const MSG_OTHER = 'Por enquanto eu entendo foto da mesa, áudio e texto. 📸
export const MSG_PHOTO_WAIT =
  '📸 Recebi a mesa! Agora me manda um áudio 🎤 ou texto contando a situação: qua
export const MSG_ANALYZING = '🎱 Analisando a mesa… já te mando as jogadas.';
export const MSG_ANALYSIS_ERROR =
  'Não consegui analisar essa foto agora. Tenta mandar novamente, de preferência
export const MSG_AUDIO_ERROR = 'Não consegui entender o áudio. 🎤 Tenta de novo f
export const MSG_BAD_IMAGE = 'Não consegui abrir essa foto. 📸 Manda de novo, em
/** Mantida por compatibilidade com a versão anterior. */
export const MSG_RECEIVED = MSG_ANALYZING;

const MAX_AUDIO_BYTES = 16 * 1024 * 1024;
const ONLY_TABLE = /^\s*(analisa|analise|analisar|vai|bora|s[oó] a mesa|l[eê] a m

export interface IncomingMessage {
  id: string;
  from: string;
  type: string;
  imageId?: string;
  imageMime?: string;
  audioId?: string;
  audioMime?: string;
  /** Texto da mensagem (ou legenda da foto). */
  text?: string;
}

export type RenderPlan = (d: Diagram, photo: Uint8Array, mime: SupportedMime) =>

export interface HandlerDeps {
  whatsapp: WhatsAppClient;
  coach: Coach;
  /** Desenha os traçados em cima da foto (opcional). */
  render?: RenderPlan;
  /** Cobrança: sem ele, todo mundo usa à vontade. */
```

```
  paywall?: Paywall;
  /** Voz (ElevenLabs) para algumas mensagens. */
  voice?: Voice;
  /** Preenchido pelo próprio handler: este número está na análise grátis. */
  free?: boolean;
}

export interface ConversationOptions {
  /** Por quanto tempo a foto fica guardada esperando a explicação. */
  photoTtlMs?: number;
  /** Por quanto tempo uma explicação mandada ANTES da foto vale pra próxima foto
  situationTtlMs?: number;
  /** Espera antes de responder um texto/áudio sem foto (caso a foto chegue logo
  askDelayMs?: number;
  now?: () => number;
}

interface Session {
  photo?: { bytes: Uint8Array; mime: SupportedMime; at: number };
  situation?: { text: string; at: number };
  pendingAsk?: { timer: ReturnType<typeof setTimeout>; text: string; heard?: stri
}

async function safeSend(whatsapp: WhatsAppClient, to: string, body: string): Prom
  try {
    await whatsapp.sendText(to, body);
    return true;
  } catch (err) {
    logger.error('Falha ao enviar mensagem', { to: maskPhone(to), ...errorInfo(er
    return false;
  }
}

/** Manda um áudio falado. Devolve false se não deu (aí quem chamou manda texto).
async function safeVoice(deps: HandlerDeps, to: string, text: string | null, cach
  if (!text || !deps.voice || !deps.whatsapp.sendAudio) return false;
  try {
    const mp3 = await deps.voice.speak(text, { cache });
    await deps.whatsapp.sendAudio(to, mp3);
    return true;
  } catch (err) {
    logger.warn('Não consegui mandar o áudio; seguindo só com texto', { to: maskP
    return false;
  }
}
```

```
const heardPrefix = (heard?: string) => (heard ? `🗣️ "${heard}"\n\n` : '');

/**
 * Conversa no WhatsApp: o jogador manda a foto e o áudio/texto (em qualquer orde
 * As mensagens de cada remetente são processadas em fila, uma de cada vez.
 */
export function createConversations(opts: ConversationOptions = {}) {
  const photoTtl = opts.photoTtlMs ?? 10 * 60_000;
  const situationTtl = opts.situationTtlMs ?? 3 * 60_000;
  const askDelay = opts.askDelayMs ?? 5_000;
  const now = opts.now ?? Date.now;
  const sessions = new Map<string, Session>();
  const queues = new Map<string, Promise<void>>();

  const session = (from: string): Session => {
    let s = sessions.get(from);
    if (!s) {
      if (sessions.size > 5_000) {
        for (const [k, v] of sessions) if (!v.pendingAsk && (!v.photo || now() -
      }
      s = {};
      sessions.set(from, s);
    }
    return s;
  };

  function enqueue(from: string, job: () => Promise<void>): Promise<void> {
    const prev = queues.get(from) ?? Promise.resolve();
    const next = prev.then(job, job).catch((err) => logger.error('Erro na fila de
    queues.set(from, next);
    void next.finally(() => {
      if (queues.get(from) === next) queues.delete(from);
    });
    return next;
  }

  async function analyze(to: string, photo: NonNullable<Session['photo']>, situat
    const { whatsapp, coach } = deps;
    await safeSend(whatsapp, to, MSG_ANALYZING);
    const started = now();
    try {
      if (coach.plan) {
        const plan = await coach.plan(photo.bytes, photo.mime, situation);
        if (plan.diagram && deps.render && whatsapp.sendImage) {
          try {
            const img = deps.render(plan.diagram, photo.bytes, photo.mime);
```

```
          if (img) await whatsapp.sendImage(to, img.png, img.caption);
        } catch (err) {
          logger.warn('Não consegui mandar a imagem das jogadas; seguindo só co
        }
      }
      await safeSend(whatsapp, to, heardPrefix(heard) + plan.text);
      await safeVoice(deps, to, choicePart(plan.text));
    } else {
      await safeSend(whatsapp, to, heardPrefix(heard) + (await coach.analyze(ph
    }
    logger.info('Análise concluída', { to: maskPhone(to), ms: now() - started,
    if (deps.free && deps.paywall) {
      deps.paywall.useFree(to);
      await safeSend(whatsapp, to, await deps.paywall.payMessage(to));
    }
  } catch (err) {
    logger.error('Falha na análise da foto', { to: maskPhone(to), ms: now() - s
    await safeSend(whatsapp, to, MSG_ANALYSIS_ERROR);
  }
}

async function answer(to: string, text: string, heard: string | undefined, deps
  if (deps.free) {
    await safeSend(deps.whatsapp, to, MSG_FREE_HINT);
    return;
  }
  if (!deps.coach.ask) {
    await safeSend(deps.whatsapp, to, MSG_TEXT);
    return;
  }
  try {
    const started = now();
    const reply = await deps.coach.ask(text);
    logger.info('Pergunta respondida pela IA', { to: maskPhone(to), ms: now() -
    await safeSend(deps.whatsapp, to, heardPrefix(heard) + reply);
  } catch (err) {
    logger.error('Falha ao responder texto', { to: maskPhone(to), ...errorInfo(
    await safeSend(deps.whatsapp, to, MSG_TEXT);
  }
}

/** Texto ou áudio já transcrito. */
async function onText(to: string, text: string, heard: string | undefined, deps
  const s = session(to);
  if (s.photo && now() - s.photo.at <= photoTtl) {
    const photo = s.photo;
```

```
    s.photo = undefined;
    await analyze(to, photo, ONLY_TABLE.test(text) ? undefined : text, heard, d
    return;
  }
  // Sem foto: guarda como situação (se a foto vier logo depois, usa) e respond
  s.situation = { text, at: now() };
  if (askDelay <= 0) {
    await answer(to, text, heard, deps);
    return;
  }
  if (s.pendingAsk) clearTimeout(s.pendingAsk.timer);
  const timer = setTimeout(() => {
    const cur = sessions.get(to);
    if (cur?.pendingAsk?.timer !== timer) return;
    cur.pendingAsk = undefined;
    void enqueue(to, () => answer(to, text, heard, deps));
  }, askDelay);
  s.pendingAsk = { timer, text, heard };
}

async function process(msg: IncomingMessage, baseDeps: HandlerDeps) {
  const to = msg.from;
  let deps = baseDeps;
  if (baseDeps.paywall) {
    let access = await baseDeps.paywall.check(to);
    // Pagou pela Hotmart mas não foi reconhecido: manda o e-mail da compra pra
    const claim = (baseDeps.paywall as Paywall & { claim?: (phone: string, text
    if (access !== 'paid' && claim && msg.type === 'text' && msg.text?.includes
      try {
        const r = await claim(to, msg.text);
        if (r === 'ok') {
          await safeSend(baseDeps.whatsapp, to, MSG_PAID);
          return;
        }
        if (r === 'notfound') {
          await safeSend(
            baseDeps.whatsapp,
            to,
            'Não achei compra aprovada com esse e-mail. 🤔 Confere se é o mesmc
          );
          return;
        }
      } catch (err) {
        logger.error('Falha ao liberar pelo e-mail', { to: maskPhone(to), ...er
      }
      access = await baseDeps.paywall.check(to);
```

```
  }
  if (access === 'blocked') {
    await safeSend(baseDeps.whatsapp, to, await baseDeps.paywall.payMessage(t
    return;
  }
  deps = { ...baseDeps, free: access === 'free' };
}
const { whatsapp, coach } = deps;

if (msg.type === 'text') {
  const text = msg.text?.trim();
  if (!text) {
    await safeSend(whatsapp, to, MSG_TEXT);
    return;
  }
  await onText(to, text, undefined, deps);
  return;
}

if (msg.type === 'audio' && msg.audioId) {
  if (!coach.transcribe) {
    await safeSend(whatsapp, to, MSG_OTHER);
    return;
  }
  let heard: string;
  try {
    const info = await whatsapp.getMediaInfo(msg.audioId);
    const bytes = await whatsapp.downloadMedia(info, MAX_AUDIO_BYTES);
    const mime = (info.mimeType ?? msg.audioMime ?? 'audio/ogg').split(';')[0
    const ext = { 'audio/ogg': 'ogg', 'audio/mpeg': 'mp3', 'audio/mp4': 'mp4'
    heard = await coach.transcribe(bytes, `audio.${ext}`);
  } catch (err) {
    logger.error('Falha ao transcrever áudio', { to: maskPhone(to), ...errorI
    await safeSend(whatsapp, to, MSG_AUDIO_ERROR);
    return;
  }
  await onText(to, heard, heard, deps);
  return;
}

if (msg.type === 'image' && msg.imageId) {
  let photo: NonNullable<Session['photo']>;
  try {
    const info = await whatsapp.getMediaInfo(msg.imageId);
    const bytes = await whatsapp.downloadMedia(info);
    photo = { bytes, mime: validateImage(bytes, info.mimeType ?? msg.imageMim
```

```
      } catch (err) {
        logger.error('Falha ao baixar a foto', { to: maskPhone(to), ...errorInfo(
        await safeSend(whatsapp, to, err instanceof MediaError ? MSG_BAD_IMAGE :
        return;
      }
      const s = session(to);
      // Legenda da foto vale como explicação.
      const caption = msg.text?.trim();
      if (caption) {
        if (s.pendingAsk) clearTimeout(s.pendingAsk.timer);
        s.pendingAsk = undefined;
        s.situation = undefined;
        await analyze(to, photo, ONLY_TABLE.test(caption) ? undefined : caption,
        return;
      }
      // Explicação que chegou antes da foto.
      if (s.pendingAsk) {
        clearTimeout(s.pendingAsk.timer);
        const { text, heard } = s.pendingAsk;
        s.pendingAsk = undefined;
        s.situation = undefined;
        await analyze(to, photo, text, heard, deps);
        return;
      }
      if (s.situation && now() - s.situation.at <= situationTtl) {
        const text = s.situation.text;
        s.situation = undefined;
        await analyze(to, photo, text, undefined, deps);
        return;
      }
      s.photo = photo;
      if (!(await safeVoice(deps, to, SPOKEN_PHOTO_WAIT, true))) await safeSend(w
      return;
    }

    await safeSend(whatsapp, to, MSG_OTHER);
  }

  return {
    /** Processa uma mensagem. Nunca lança. */
    handle(msg: IncomingMessage, deps: HandlerDeps): Promise<void> {
      return enqueue(msg.from, () => process(msg, deps));
    },
  };
}
```